

PointNet meets Self-Attention Graph Pooling: A Synergistic Approach to Point Cloud Classification

1st M. Ebadpour

Master of Science student
Amirkabir University of Technology
Tehran, Iran
M.Ebadpour@aut.ac.ir

2nd M. Choupan

Master of Science student
University of Bologna
Bologna, Italy
mohammad.choupan@studio.unibo.it

3rd M. Javanmardi

Assistant Professor
Amirkabir University of Technology
Tehran, Iran
MJavan@aut.ac.ir

Abstract—The point cloud is a crucial 3D data structure that plays a significant role in environmental research and has extensive applications in industries such as autonomous driving and image segmentation. Additionally, graphs are useful data structures for regularizing point cloud data, allowing the incorporation of graph features to extract specific attributes for each point in the dataset. Graph-based neural networks facilitate the extraction of features from point cloud data, ensuring that each point’s unique characteristics contribute to the classification results, thereby enhancing detection accuracy. In this paper, we investigate the impact of graph centrality features on each point using PointNet and Self-Attention Graph Pooling as baselines. We also introduce two novel architectures for point cloud classification. The first architecture integrates the Point-Net and Self-Attention Graph Pooling models using fusion approaches. The second model is a Point-Net based Self-Attention Graph Pooling, which utilizes the results from the first model as input for the second. Extensive experiments conducted on the ModelNet10 dataset demonstrate that Closeness and Harmonic centralities are the best centralities for classification. Our new architectures improve classification accuracy by up to 2.85% compared to the PointNet baseline, while also reducing the model size by over 1.1MB.

Index Terms—Point Cloud Classification, PointNet, Self-Attention Graph Pooling, 3D Object Recognition.

I. Introduction

In recent years, 3D computer vision has become an essential component of many real-world applications, including autonomous driving, robotics, virtual and augmented reality, and digital heritage. Among the different 3D representations, point clouds have emerged as a particularly flexible and expressive data format. A point cloud represents an object or scene as an unordered set of points in $\mathbb{R}^3$, typically described by their (x, y, z) coordinates and, in some cases, additional attributes such as color or intensity. Despite their flexibility, point clouds pose significant challenges for traditional deep learning architectures, which are usually designed for grid-structured data such as images or regular volumes.

Early approaches to point cloud analysis often relied on hand-crafted features, voxelization, or multi-view projections. Voxel-based methods discretize the 3D space into a regular grid, enabling the use of 3D convolutional neural networks, but they suffer from substantial memory

overhead and loss of geometric detail due to resolution constraints. Multi-view methods project the 3D object onto a set of 2D images and then apply standard 2D CNNs; while effective in some cases, they may discard important 3D structural information and often require complex rendering pipelines. These limitations motivated the development of architectures that operate directly on raw point sets.

PointNet [1] introduced a breakthrough formulation for point cloud classification and segmentation by processing each point independently with shared multilayer perceptrons (MLPs) and aggregating local features using a symmetric function (e.g., max pooling). This design guarantees invariance to the ordering of points and enables efficient learning directly in the point domain. However, PointNet and many of its variants focus primarily on per-point features and global pooling, and therefore have limited ability to explicitly model the relational structure and long-range dependencies among points. As a result, they may struggle in scenarios where the geometric configuration and interactions between different parts of an object are critical for accurate recognition.

Graphs offer a natural way to encode the relational structure of point clouds. By treating each point as a node and connecting nearby points with edges, a point cloud can be converted into a graph in which both local neighborhoods and global connectivity patterns are explicitly represented. Graph Neural Networks (GNNs) and graph pooling methods can then be used to propagate information along edges and to build hierarchical representations of the underlying 3D shape. In particular, self-attention-based pooling schemes such as Self-Attention Graph Pooling (SAGPool) learn to assign importance scores to nodes and select the most informative ones, effectively focusing the model on salient regions of the object while reducing computational cost.

Beyond the raw geometry, graphs also make it possible to compute rich structural descriptors such as centrality measures. Centrality metrics like betweenness, closeness, eigenvector, harmonic, Katz, load, and PageRank centralities quantify different notions of importance or

influence of nodes within the graph. When computed on graphs derived from point clouds, these measures capture how each point is embedded in the global structure of the object, for example whether it lies on a highly connected hub, along a critical path, or on the periphery of the shape. Such information is complementary to the local geometric cues typically exploited by point-based networks.

These observations motivate the development of fusion architectures that combine the strengths of point-based and graph-based models. PointNet-style networks excel at capturing local geometric patterns and learning robust global descriptors from unordered point sets, while SAGPool-style graph networks are well-suited to modeling relational structure and focusing on the most informative regions of a graph. By integrating point-wise features extracted by PointNet with graph-level features produced by SAGPool, and by enriching both with centrality-based descriptors, it is possible to construct more expressive and discriminative representations for point cloud classification.

A. Traditional Point Cloud Networks and Their Limitations

Traditional point cloud classification networks such as PointNet and its variants directly consume unordered point sets and leverage symmetric aggregation functions to ensure permutation invariance. These models have demonstrated strong performance on benchmarks like ModelNet10 and ModelNet40, and they represent a key milestone in 3D deep learning. However, because each point is processed independently before global pooling, the explicit pairwise or higher-order relationships among points are not fully exploited. This can limit the ability of such networks to distinguish between shapes that share similar local patterns but differ in their global arrangement.

Another limitation concerns scalability and generalization. As the number of points or the complexity of the object increases, capturing all relevant interactions through simple global pooling becomes more difficult. In addition, while PointNet employs transformation networks (T-Nets) to align the input and intermediate features, it still relies on learned per-point embeddings that may not capture long-range dependencies and global context as effectively as graph-based methods.

B. Graph-Based Methods and SAGPool Architecture

Graph-based methods address these limitations by explicitly modeling the relationships among points. Once a graph is constructed from a point cloud (for example, using k -nearest neighbors in Euclidean space), Graph Neural Networks can propagate information along edges to aggregate features from local neighborhoods. Stacking multiple graph convolutional layers yields receptive fields that grow with depth, enabling the network to capture both local and global structure.

Self-Attention Graph Pooling (SAGPool) [2] builds on this idea by introducing a learnable attention mechanism for graph pooling. Instead of heuristically downsampling nodes, SAGPool predicts an importance score for each node and retains only the most informative ones, discarding less relevant points and their incident edges. This leads to compact yet expressive graph representations that emphasize salient regions of the object, improving both accuracy and computational efficiency in graph classification tasks. Nonetheless, when SAGPool is applied directly to point clouds, it may underutilize rich point-wise geometric features and may not fully exploit the powerful feature extractors developed in point-based networks.

C. Motivation for Fusion Approaches

The complementary strengths of PointNet and SAGPool suggest that a carefully designed fusion of point-based and graph-based representations can yield more powerful models for point cloud classification. PointNet provides strong per-point descriptors and a robust global summary of the point set, while SAGPool contributes hierarchical graph representations and attention-driven node selection. Moreover, centrality measures computed on point cloud graphs encode additional structural information that is not directly accessible to purely point-based architectures.

In this work, we exploit these synergies by proposing fusion networks that integrate PointNet and SAGPool at the feature level. Our architectures ingest both raw coordinates and centrality features, construct graphs over the point clouds, and combine the resulting point-wise and graph-wise representations in a unified framework. The goal is to improve classification accuracy while preserving or even reducing memory footprint compared to standard baselines, making the models suitable for deployment in resource-constrained settings.

D. Contributions

To summarize, this paper makes the following contributions:

- We propose two novel fusion architectures, FeatureConcat and Graph Classification Based on PointNet Features (PointNetBasedGraphPooling), which jointly exploit point-wise features from PointNet and graph-based representations from SAGPool for point cloud classification.
- We introduce a comprehensive feature engineering pipeline that augments raw 3D coordinates with multiple graph-based centrality measures, enabling a richer description of the structural role of each point in the underlying graph.
- We present an efficient GPU-based implementation and extensive experiments on the ModelNet10 and ModelNet40 datasets, providing a detailed analysis of accuracy, model size, and generalization, and demonstrating that the proposed fusion models

outperform the individual PointNet and SAGPool baselines.

II. Related Works

Research on 3D point cloud understanding has progressed along two main lines: methods that operate directly on unordered point sets, and methods that first convert point clouds into structured representations such as graphs, voxels, or multi-view images. In this section, we briefly review the most relevant approaches and position our work with respect to them.

A. Point-Based Deep Networks for Point Clouds

PointNet [1] pioneered end-to-end learning directly on raw point sets by processing each point independently with shared multilayer perceptrons and aggregating features using a symmetric function to guarantee permutation invariance. Despite its simple architecture, PointNet achieved strong performance on benchmarks such as ModelNet10 and ModelNet40 and became a cornerstone for subsequent work on 3D deep learning.

PointNet++ [3] extended this idea by introducing a hierarchical feature-learning scheme that groups points into local neighborhoods at multiple scales, enabling the network to capture local geometric details while retaining the global representation power of PointNet. Other architectures, such as PointCNN [11], KPConv [12], and EdgeConv-based Dynamic Graph CNNs for learning on point clouds [4], further refined local feature extraction by dynamically constructing graphs in the feature space and applying edge convolutions that are sensitive to local geometry. These point-based models have shown that learning directly from unordered point sets is both feasible and effective, but they typically rely on global pooling operations and do not explicitly encode long-range relational structure among points.

B. Graph-Based Methods and Pooling on Point Clouds

Graphs provide a natural framework for modeling the relational structure of point clouds. Once a graph is constructed (for example, using k -nearest neighbors in Euclidean space), Graph Neural Networks can be applied to propagate information along edges. Early work on spectral graph convolutional networks and signal processing on graphs [5]–[8] laid the foundation for learning filters in the spectral domain of the graph Laplacian, while later spatial methods defined convolutions directly in the vertex domain.

In the context of point clouds, graph representations have been used to capture both local neighborhoods and global topology, often leading to improved robustness to irregular sampling and noise. A key challenge is how to perform pooling on graphs, since standard downsampling operators used in CNNs do not directly transfer to irregular structures. Various graph pooling strategies have been proposed, including clustering-based coarsening,

top- k selection [10], and attention-based mechanisms. Self-attention graph pooling approaches, such as the SAGPool architecture considered in this work, learn importance scores for nodes and retain only the most informative ones, yielding compact and expressive graph-level representations while discarding less relevant nodes and edges.

C. Fusion of Point-Based and Graph-Based Representations

While point-based and graph-based methods each have specific advantages, a single representation often cannot fully exploit both local geometric cues and global structural information. Several works therefore attempt to combine point features with graph-based reasoning. Some architectures first compute point-wise descriptors using PointNet-like blocks and then build graphs in the feature space to capture contextual relations among points; others interleave graph convolutions with point-based operations to refine features at multiple scales.

Our work follows this line of research but focuses on explicit fusion between a PointNet backbone and a hierarchical graph pooling network (SAGPool). The proposed FeatureConcat and PointNetBasedGraphPooling architectures are designed to exploit complementary strengths: PointNet provides strong point-wise embeddings and global descriptors, while SAGPool contributes hierarchical graph-level representations and attention-driven node selection. Compared to methods that rely exclusively on either point-based or graph-based processing, our fusion schemes aim to achieve higher accuracy with a competitive or even reduced memory footprint.

D. Centrality Measures and Graph-Derived Features

Graph-based centrality measures such as betweenness, closeness, eigenvector, harmonic, Katz, load, and PageRank centralities have been widely studied in network analysis [13], [14]. They quantify different notions of importance or influence of nodes within a graph, capturing how each node contributes to global connectivity, information flow, or structural cohesion. In many applications, centrality features have been used as powerful descriptors for tasks such as community detection, anomaly detection, and classification.

In the context of point clouds, centrality measures computed on graphs derived from 3D shapes provide complementary information to raw coordinates and learned features. For example, points on object boundaries, thin structures, or junctions may exhibit distinctive centrality patterns that help distinguish similar shapes. However, relatively few works have systematically investigated the use of such graph-derived features in combination with deep point cloud networks.

The present work contributes to this direction by integrating multiple centrality measures into both point-based and graph-based pipelines. We treat centralities as

additional per-point channels and study their impact on PointNet, SAGPool, and two fusion architectures. Our experiments on ModelNet10 and ModelNet40 demonstrate that carefully designed fusion of point-wise, graph-based, and centrality-based features can significantly improve point cloud classification performance over strong baselines.

III. PROPOSED APPROACHES

A. Feature Engineering

In this section, we present our approach to feature engineering in point cloud classification due to the limitations of XYZ features. To enhance feature representation, we exploit graph-based features, starting with the construction of a weighted and directed graph using the k-nearest neighbors method. As we consider the inherent 3D spatial relationships, we set K to 6, representing the six absolute sides in 3D space. This graph enables efficient feature aggregation and captures crucial spatial dependencies among points.

To further enrich the feature set, we introduce node centrality as an essential feature for each node (point in space) within the graph. We employ seven centrality metrics, namely betweenness, closeness, Katz, PageRank, eigenvector, harmonic, and load centrality.

- **Betweenness:** This centrality feature measures the importance of a node in connecting other nodes. Mathematically, let $n_{s,t}^i$ be the number of geodesic paths from s to t that pass through i and let $n_{s,t}$ be the total number of geodesic paths from s to t . Recall that a geodesic path is not necessarily unique and the geodesic paths between a pair of vertices need not be node-independent, meaning they may pass through some of the same vertices. Then the betweenness centrality of vertex i is calculated by equation 1:

$$b_i = \sum_{s,t} w_{s,t}^i = \sum_{s,t} \frac{n_{s,t}^i}{n_{s,t}} \quad (1)$$

- **Closeness:** closeness centrality (or closeness) of a node is a measure of centrality in a network, calculated as the sum of the length of the shortest paths between the node and all other nodes in the graph. Thus the more central a node is, the closer it is to all other nodes. The closeness centrality of node x is calculated by equation 2. where $d(y, x)$ is the distance between vertices x and y .

$$C(x) = \frac{N}{\sum_y d(y, x)} \quad (2)$$

- **Katz:** Katz centrality computes the relative influence of a node within a network by measuring the number of the immediate neighbors (first degree nodes) and also all other nodes in the network that connect to the node under consideration through these immediate

neighbors. In summary it combines both local and global node information. For calculate Katz for a node Let $A = (a_{i,j})$ be the adjacency matrix of a directed graph. The Katz centrality x_i of node i is given by:

$$x_i = \alpha \sum_k a_{k,i} x_k + \beta$$

where α and β are constants. In matrix form we have:

$$x = \alpha x A + \beta$$

where β is now a vector whose elements are all equal a given positive constant. The free parameter α , known as the damping factor, should be chosen with case. For small (close to 0) values of α the contribution given by paths longer than one rapidly declines, and thus Katz scores are mainly influenced by short paths (mostly in-degrees) and by the exogenous factor of the system.

- **PageRank:** PageRank is an adjustment of Katz centrality that takes into consideration this issue. There are three distinct factors that determine the PageRank of a node: (i) the number of links it receives, (ii) the link propensity of the linkers, and (iii) the centrality of the linkers. The first factor is not surprising: the more links a node attracts, the more important it is perceived. For calculate PageRank for a node Let $A = (a_{i,j})$ be the adjacency matrix of a directed graph. The PageRank centrality x_i of node i is given by:

$$x_i = \alpha \sum_k \frac{a_{k,i}}{d_k} x_k + \beta$$

where α and β are constants and d_k is the out-degree of node k if such degree is positive, or $d_k = 1$ if the out-degree of k is null. In matrix form we have:

$$x = \alpha x D^{-1} A + \beta$$

where β is now a vector whose elements are all equal a given positive constant and D^{-1} is a diagonal matrix with i -th diagonal element equal to $1/d_i$. Notice that, as seen for Katz centrality, PageRank is determined by an endogenous component that takes into consideration the network topology and by an exogenous component that is independent of the network structure. It follows that x can be computed as:

$$x = \beta(I - \alpha D^{-1} A)^{-1}$$

The damping factor α and the personalization vector β have the same role seen for Katz centrality. In particular, α should be chosen between 0 and $1/\rho(D^{-1}A)$. Also, when the network is large, it is preferable to use the power method for the computation of PageRank.

- **Eigenvector:** Eigenvector centrality (also called eigencentality) is a measure of the influence of a node in a network. It assigns relative scores to all nodes in

the network based on the concept that connections to high-scoring nodes contribute more to the score of the node in question than equal connections to low-scoring nodes. PageRank and the Katz centrality are variants of the eigenvector centrality. Using the adjacency matrix to find eigenvector centrality For a given graph $G := (V, E)$ with $|V|$ vertices let $A = (a_{v,t})$ be the adjacency matrix, i.e. $a_{v,t} = 1$ if vertex v is linked to vertex t , and $a_{v,t} = 0$ otherwise. The relative centrality score of vertex v can be defined as:

$$x_v = \frac{1}{\lambda} \sum_{t \in M(v)} , x_t = \frac{1}{\lambda} \sum_{t \in G} a_{v,t} x_t$$

- **Harmonic:** Harmonic centrality measures the 'average' distance of a node to the other nodes in the network. This can be used to find nodes in a network that can quickly communicate with other nodes. For calculation harmonic centrality for node i let $d_{i,x}$ is a distance from node i to node x for a given graph $G := (V, E)$ with $|V|$ vertices then :

$$Harmonic(i) = \sum_{x \in G, x \neq i} \frac{1}{d_{i,x}}$$

- **Load centrality:** The load centrality of a node is the fraction of all shortest paths that pass through that node. It measures the node's importance in graph connectivity and load distribution. For calculate load centrality Consider a graph $G := (V, E)$ and an algorithm to identify the (potentially multiple) minimum weight path(s) between any pair of vertices (s, d) . Let $\theta_{s,d}$ be a quantity of a commodity that is sent from vertex s to vertex d . We assume the commodity is always passed to the next hop following the minimum weight paths. In case of multiple next hops, the commodity is divided equally among them. We call $\theta_{(s,d)}(v)$ the amount of commodity forwarded by vertex v . The load centrality of v is then given by:

$$LC(v) = \sum_{s,d \in V} \theta_{(s,d)}(v)$$

In our feature engineering process, we combine the seven centrality features with the existing XYZ features (x, y, and z coordinates) for each node. This results in a feature vector comprising 10 distinctive elements per point in the point cloud. Additionally, we applied a 1024 sampling rate to each point cloud to handle varying point densities efficiently. To ensure uniformity, we performed normalization across the feature vectors. Furthermore, random noise was introduced to augment the dataset, enhancing the model's robustness to real-world scenarios and improving generalization performance. These feature engineering techniques aim to provide a comprehensive and informative representation of the point cloud data, optimizing the classification performance in the subsequent stages of our research. As shown in the

subsequent section, the features closeness and harmonic are identified as the most effective additions to the point cloud's existing x, y, and z dimensions. These features were subsequently utilized to advance our approach.

The calculation of feature vectors for node centralities was implemented using multiprocessing on the CPU. This approach allowed us to compute centrality metrics for each node in the point cloud concurrently, significantly reducing the computation time. Using multiprocessing, we accelerated the feature engineering process and handled large datasets more efficiently.

We selected Point-Net and Self-Attention Graph Pooling models as baselines for improvement. Point-Net is a general benchmark model for point cloud classification, suggesting that any enhancements achieved with this baseline could potentially be replicated with other models. Similarly, Self-Attention Graph Pooling is a popular benchmark in graph classification, and achieving better results with this model could indicate improved generalization across other baselines.

B. Feature Concatenation Approach

In this proposed fusion approach, named "FeatureConcatModel," we combine the high-level features obtained from PointNet and SAGPool networks for efficient point cloud classification. Both networks take point clouds with 10 defined features as input. SAGPool outputs vectors with 64 features for each point cloud in a batch.

To adapt PointNet, we introduce a linear layer, reducing the 1024 features to 64 features for each point cloud. Subsequently, we concatenate the 64-feature vectors from both PointNet and SAGPool, creating a unified vector with 128 features. This concatenated feature vector captures both the local and global information, ensuring a comprehensive representation of the point cloud data. To further process the concatenated features, we employ three multi-layer perceptron (MLP) networks. The first MLP contains 64 neurons, followed by a second MLP with 32 neurons, and finally a third MLP with 10 neurons corresponding to the number of classes. Batch normalization is applied to all three MLPs, enhancing the model's stability and accelerating convergence.

We employ dropout techniques with a probability of 0.3 in the second layer, reducing overfitting and improving generalization performance. Additionally, we incorporate the Graph Attention (GAT) layer from SAGPool with 6 attention heads, considering the 6 nearest neighbors for each point.

The architecture for feature concatenation is presented in Fig. 1.

C. Graph Classification based on PointNet Features

The "PointNetBasedGraphPoolingModel" fusion approach integrates SAGPool and PointNet architectures. To begin, we utilize a set of 10 defined features,

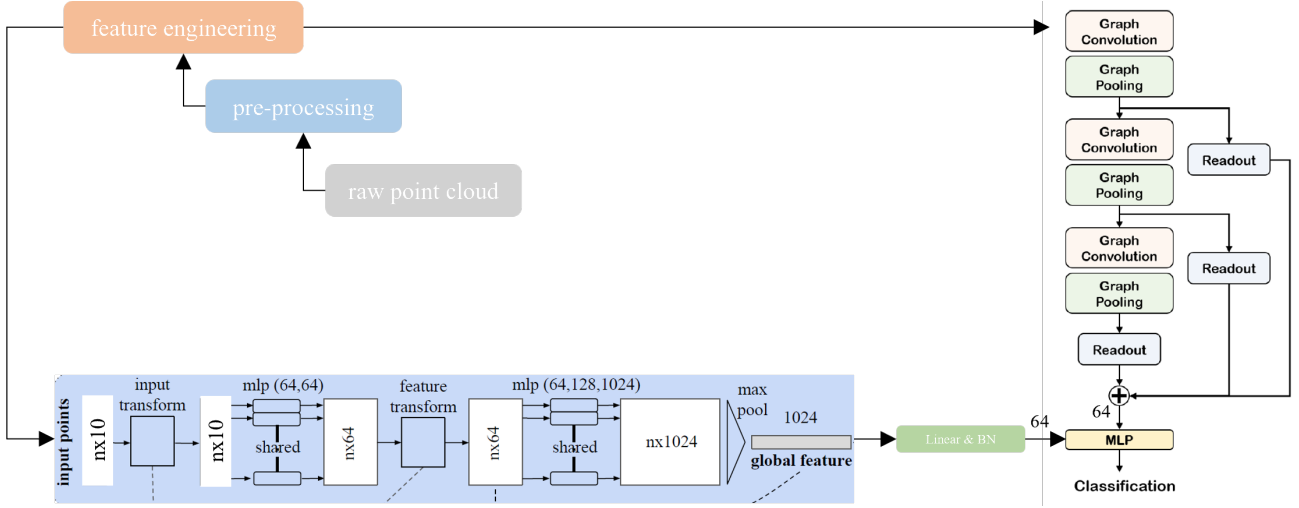


Fig. 1. The architecture for Feature Concatenation Approach.

including XYZ coordinates and centrality metrics, as input to the custom version of PointNet. In this custom implementation, we intentionally omit the global pooling section to obtain a 1024-feature vector for each node in the point cloud. This design choice enables PointNet to capture local features and spatial relationships present in the data. The architecture for the graph classification based on PointNet features is shown in Fig. 2.

Subsequently, we reduce the dimensionality of the feature vectors obtained from PointNet to 32 by applying a 1D convolution, followed by batch normalization. The resulting 32-feature vectors, encoding local feature representations, are then fed into the SAGPool network.

SAGPool efficiently performs adaptive feature pooling and aggregation. The integration of the PointNet-derived feature vectors helps the network aggregate global context while retaining local details.

Experimental evaluations of the "PointNetBasedGraphPoolingModel" approach show strong performance in point cloud classification. By combining the local feature extraction capabilities of PointNet with the adaptive feature aggregation of SAGPool, the approach achieves a richer representation of the point cloud data. The fusion of these two architectures leads to notable improvements in classification accuracy, enabling more accurate recognition while reducing the model size.

IV. EXPERIMENTS AND RESULTS

A. Implementation details and hyper-parameters

In our implementation, we adopted the PyTorch framework as the primary tool for model development. To facilitate graph pooling and centrality calculations, we incorporated PyTorch Geometric and NetworkX libraries, respectively. A comprehensive dataloader was designed, encompassing all

essential details, including XYZ features, graph edge lists, centrality features, and labels.

To optimize processing costs and expedite training, we employed parallel computation to precalculate and store the required features in separate files. During training, we efficiently loaded the precomputed features from the drive, minimizing redundant calculations and streamlining the training process.

For training and testing, we set the batch size to 32 and 64, respectively. Across all networks, a graph pooling rate of 0.25 was consistently applied, along with a Graph Attention (GAT) convolution layer featuring 6 heads and 128 hidden features.

The ADAM optimizer was chosen for training, with weight decay 0.0005 and learning rates of 0.01, 0.001, and 0.005 for the SAGPool, PointNet, and fusion approaches, respectively. Due to resource limitations during the research, we trained the networks for 60-200 epochs, allowing us to explore a wide range of model performance within our constraints.

Our experiments were conducted on a Desktop PC with a robust hardware configuration. The machine featured an Intel Core i7 12700K CPU, operating at 4.9 GHz frequency, and 32GB DDR5 main memory. Additionally, an NVIDIA RTX 3090 GPU with 24GB memory was utilized for accelerated computations.

In summary, our implementation details and well-optimized hyperparameters ensured efficient and effective exploration of the fusion approaches' performance. While resource constraints influenced certain aspects of our experiments, the chosen hardware configuration allowed us to achieve substantial progress in understanding the potential benefits of combining SAGPool and PointNet architectures for point cloud classification tasks.

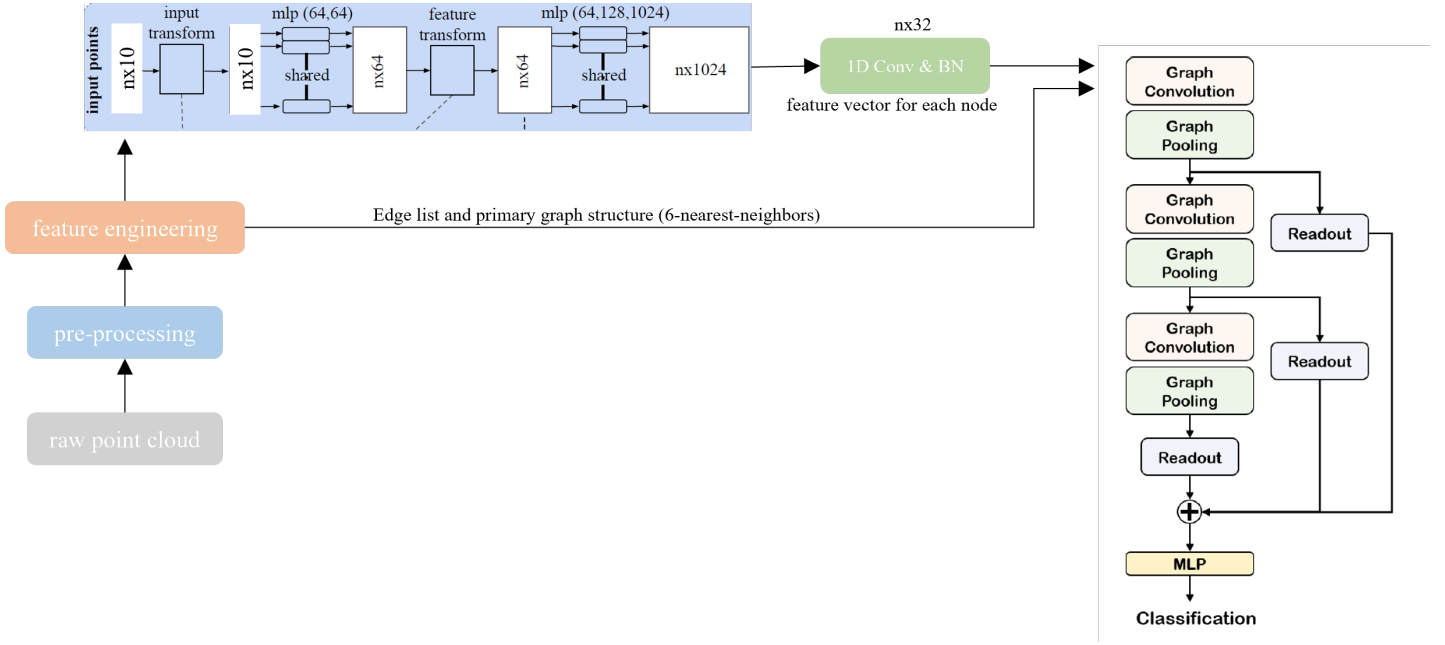


Fig. 2. The architecture for the graph classification based on PointNet features.

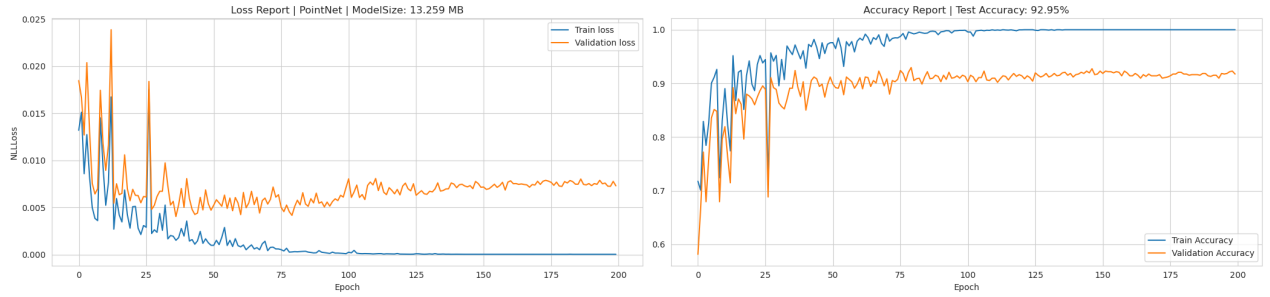


Fig. 3. Training result for traditional PointNet Network On model Net 10.

B. Dataset

The dataset utilized for training and testing the networks in this study is ModelNet, a widely used benchmark in the field of 3D object recognition and classification. ModelNet comprises ModelNet10 and ModelNet40, which feature 10 and 40 classes, respectively. These datasets include categories of common objects such as chairs, tables, airplanes, and cars, among others. To prepare the dataset for training and testing, several preprocessing steps were applied. Firstly, we employed a 1024 sampling rate to obtain a fixed number of points from each 3D model. This step ensured a consistent input size for the networks, facilitating efficient processing. Next, we applied normalization to scale the point coordinates, making them relative to the model's bounding box and ensuring that they lie within a common range. This normalization step enhanced the model's robustness and improved convergence during training.

Additionally, to augment the dataset and enhance the model's generalization capabilities, we introduced

random noise to the point cloud data. This noise addition introduced small variations in the point positions, mimicking real-world scenarios and improving the network's ability to handle noisy input data. Furthermore, random rotation was applied to the 3D models during training. This augmentation technique involved rotating the objects along different axes, creating diverse orientations for each model. By doing so, the network learned to be invariant to object rotations, improving its ability to recognize and classify objects from different viewpoints.

To ensure an unbiased evaluation of the models' performance, the original ModelNet dataset was divided into predefined training and testing sets. The training set was used to train the fusion approaches, while the testing set was kept separate for evaluation. This separation allowed for an objective assessment of the models' classification accuracy and generalization to unseen data. The utilization of the ModelNet10 dataset, combined with the various preprocessing and augmentation techniques,

contributed to the robustness and effectiveness of the trained networks in point cloud classification tasks.

C. Training

In this section, we provide a detailed account of the training process for the four networks aimed at investigating the performance of the intersection of SAGPool and PointNet compared to each architecture used individually. For the first network, a traditional PointNet was trained using the standard ModelNet10 dataset with 3 features (XYZ coordinates). The results of this training are presented in Fig. 3.

The second network involved a hierarchical version of SAGPool with a custom improvement that incorporated learning weights for each stage. We trained this network using a feature vector of 10 dimensions, which included the XYZ coordinates along with centrality metrics. The training outcomes for this hierarchical SAGPool network can be observed in Fig. 4.

Additionally, we examined the impact of each feature on the PointNet and Self-Attention Graph Pooling baselines, with results presented in Table I and Table II.

These results demonstrate that Harmonic centrality is the most effective feature in the Self-Attention Graph Pooling model for both ModelNet10 and ModelNet40 datasets. This is because Harmonic centrality enhances the importance of points located on the edges of the point cloud data, which, followed by other features, yields the second-best result in this mode.

Conversely, in the Point-Net model, Katz centrality emerges as the best feature added to the base feature set. This is attributed to the model’s T-net and feature extraction layer, which significantly influence shape recognition. Katz Centrality specifically enhances feature extraction at the edges, similar to the Self-Attention Graph Pooling model. However, the primary difference between these two models lies in their architecture and their distinct approaches to impacting the data.

The third network implemented our proposed feature concatenation approach, where we combined the high-level features extracted from both PointNet and SAGPool networks.

TABLE I

Training results of each feature on PointNet on ModelNet10. The features used during training and test accuracy achieved are reported.

Features	Acc
3 (XYZ)	91.85
3 (XYZ) + Beetweenness	92.29
3 (XYZ)+ Closeness	92.73
3 (XYZ)+Eigenvector	92.51
3 (XYZ)+Harmonic	92.29
3 (XYZ)+Katz	92.62
3 (XYZ)+Load centrality	92.29
3 (XYZ) + PageRank	92.4
All Features	92.95

TABLE II

Training results of each feature on the Self-Attention Graph Pooling model on ModelNet10. The features used during training and test accuracy achieved are reported.

Features	Acc
3 (XYZ)	87.11
3 (XYZ) + Beetweenness	86.12
3 (XYZ)+ Closeness	83.26
3 (XYZ)+Eigenvector	87.33
3 (XYZ)+Harmonic	87.89
3 (XYZ)+Katz	83.04
3 (XYZ)+Load centrality	87.56
3 (XYZ) + PageRank	87.22
All Features	87.78

Similarly, a feature vector of 10 dimensions, encompassing XYZ coordinates and centrality metrics, was used for training. The training results for this feature concatenation approach are depicted in Fig. 5. Lastly, we explored the graph classification based on PointNet features approach as our fourth network. Again, a feature vector of 10 dimensions, comprising XYZ coordinates and centrality metrics, was utilized for training. The training outcomes for this graph-based classification approach are illustrated in Fig. 6. In all four networks, the same sets were used for training and testing, ensuring a fair and unbiased evaluation of their performance. The inclusion of centrality metrics in the feature vectors enhances the models’ capability to capture essential node connectivity

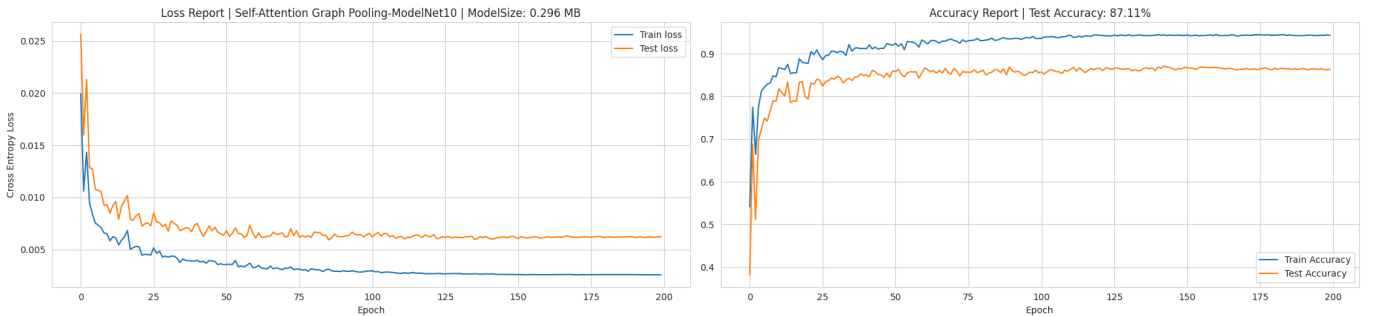


Fig. 4. Training result for original hierarchical SAGPool network On model Net 10.

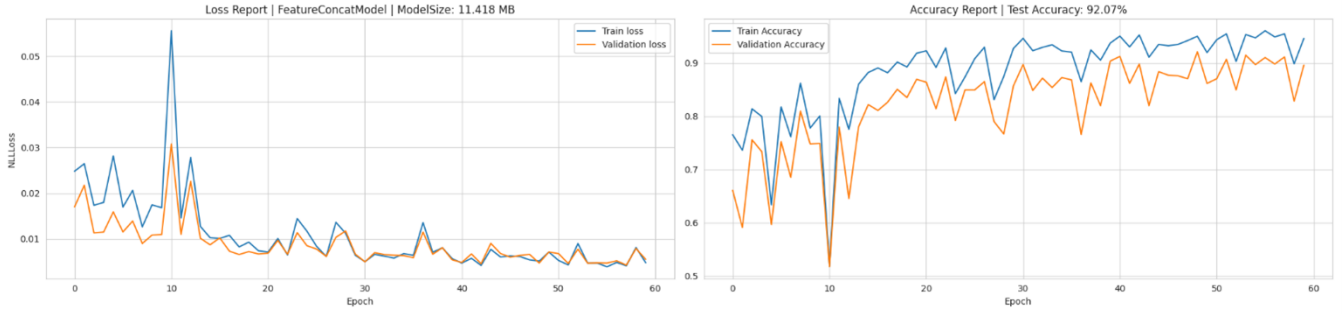


Fig. 5. Training result for feature concatenation approach On model Net 10.

TABLE III

Training results of each feature on the Self-Attention Graph Pooling (SAGPool) network on ModelNet40. The learning rate is fixed to 0.004 and the weight decay to 0.005.

Features	Acc (%)
3 (XYZ)	78.44
3 (XYZ) + Betweenness	74.11
3 (XYZ) + Closeness	74.80
3 (XYZ) + Eigenvector	68.48
3 (XYZ) + Harmonic	77.03
3 (XYZ) + Katz	71.47
3 (XYZ) + Load	71.80
3 (XYZ) + PageRank	71.27
All features (XYZ + 7 centralities)	75.04

information, contributing to improved classification accuracy. These comprehensive experiments allow us to gain valuable insights into the efficacy of combining SAGPool and PointNet architectures and highlight their potential to enhance point cloud classification tasks. The results obtained from these experiments play a pivotal role in shaping our understanding of the proposed fusion approaches and their practicality in real-world applications. Also, a summary of the training results is presented in Table IV.

D. Convergence analysis and final quantitative results

The learning curves in Figs. 3–6 show that all four architectures converge stably on ModelNet10. The

training loss decreases smoothly and the validation accuracy quickly plateaus, with no evidence of divergent behaviour. The SAGPool and PointNet baselines reach their best validation accuracy well before the last epoch, while the two fusion approaches (FeatureConcat and PointNetBasedGraphPooling) benefit from the full training schedule, showing small but consistent gains in the final epochs.

On ModelNet40, the optimization dynamics are similar, although the task is more challenging and all models converge to a lower absolute accuracy. For the Self-Attention Graph Pooling (SAGPool) baseline, Table II reports the effect of injecting a single centrality on ModelNet10, while Table III extends the same analysis to ModelNet40. On ModelNet10, harmonic centrality yields the best performance (87.89%), closely followed by load and eigenvector centralities, whereas concatenating all seven centralities with XYZ achieves 87.78%. On ModelNet40, the best configuration with a single feature type is obtained when using only the XYZ coordinates (78.44%), while concatenating all seven centralities with XYZ reaches 75.04%. These results indicate that centrality information is more beneficial on the smaller ModelNet10 benchmark, while on ModelNet40 the graphs become denser and the additional features are harder to exploit effectively.

For the PointNet baseline, hyper-parameter tuning on ModelNet10 shows that using a learning rate of 0.02 and weight decay of 10^{-4} leads to the best overall performance

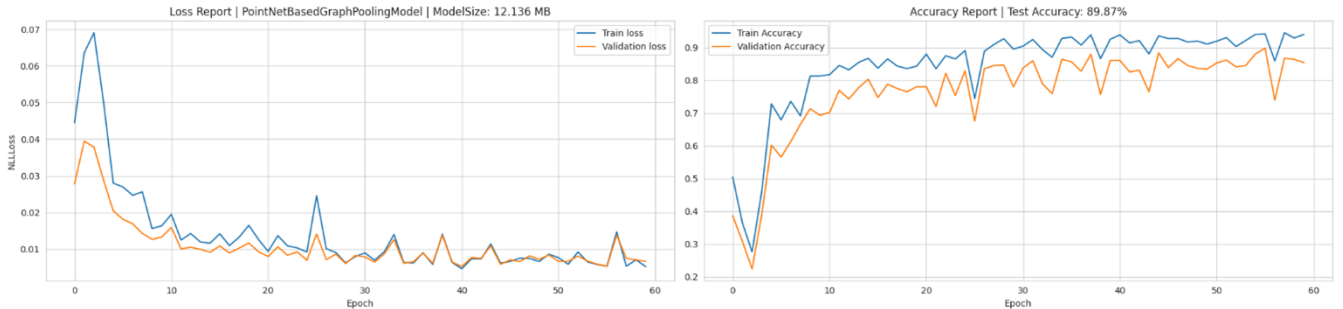


Fig. 6. Training result for graph classification based on PointNet features approach On model Net 10.

TABLE IV

Best test accuracy on ModelNet10 and ModelNet40 for the four network architectures. For each model we report the best-performing configuration in terms of input features. Model sizes are reported for the corresponding PyTorch implementations.

Network	Dataset	Features	Acc (%)	Size
PointNet	ModelNet10	3 (XYZ) + 7 (Centralities)	92.95	13.259 MB
PointNet	ModelNet40	3 (XYZ)	87.44	13.259 MB
Hierarchical SAGPool	ModelNet10	3 (XYZ) + Harmonic	87.89	0.3 MB
Hierarchical SAGPool	ModelNet40	3 (XYZ)	78.44	0.3 MB
FeatureConcat	ModelNet10	3 (XYZ)	94.34	11.418 MB
FeatureConcat	ModelNet40	3 (XYZ) + Harmonic	88.20	11.418 MB
PointNetBasedGraphPooling	ModelNet10	3 (XYZ) + 7 (Centralities)	95.80	12.136 MB
PointNetBasedGraphPooling	ModelNet40	3 (XYZ) + Harmonic	85.47	12.136 MB

when all features are used, yielding 92.95% accuracy. When we inject a single centrality at a time, the best result is obtained with closeness centrality (92.73%), followed by Katz centrality (92.62%). On ModelNet40, the best PointNet configuration with XYZ input reaches 87.44% accuracy.

The two proposed fusion architectures clearly benefit from combining PointNet and SAGPool features. For the FeatureConcat model, using only XYZ features yields 94.34% accuracy on ModelNet10 and 87.60% on ModelNet40, while concatenating XYZ with all seven centralities achieves 92.29% and 86.67% on ModelNet10 and ModelNet40, respectively. When harmonic centrality is used as the only graph-derived feature, the corresponding accuracies are 92.26% (ModelNet10) and 88.20% (ModelNet40).

For the PointNetBasedGraphPooling approach, the configuration using all features (XYZ plus the seven centralities) achieves our best overall result on ModelNet10, with 95.80% test accuracy, and 84.07% on ModelNet40. When only harmonic centrality is added to XYZ, the accuracies are 93.74% (ModelNet10) and 85.47% (ModelNet40). These experiments confirm that harmonic centrality is one of the most informative graph-derived features in our setting, and that the proposed fusion schemes consistently outperform the individual baselines across both datasets.

V. CONCLUSION

A. Networks Comparison

In terms of model size, the Hierarchical SAGPool network stands out as the most compact model with approximately 0.3 MB of parameters. The two fusion architectures, FeatureConcat and PointNetBasedGraphPooling, require 11.418 MB and 12.136 MB respectively, while PointNet has the largest footprint with 13.259 MB.

From the perspective of test accuracy, the proposed fusion approaches clearly outperform the baselines. On ModelNet10, PointNetBasedGraphPooling achieves the best overall performance with 95.80% accuracy when trained on all features (XYZ coordinates and the seven centrality measures). The FeatureConcat architecture attains 94.34% accuracy using only the XYZ features,

while the best PointNet configuration reaches 92.95% with all features. The hierarchical SAGPool baseline obtains 87.89% accuracy when harmonic centrality is added to the XYZ coordinates.

On the more challenging ModelNet40 dataset, the relative ranking of the architectures is similar, although all networks achieve lower absolute accuracy. The highest accuracy is obtained by the FeatureConcat model with 88.20% when using XYZ and harmonic centrality. PointNetBasedGraphPooling reaches 85.47% with XYZ plus harmonic centrality, the best PointNet configuration achieves 87.44% with XYZ input, and SAGPool attains 78.44% with XYZ features.

Regarding generalization, these results show that combining PointNet and SAGPool features in a single fusion architecture yields a better accuracy–complexity trade-off than either baseline alone. PointNetBasedGraphPooling is consistently the strongest model on ModelNet10, while FeatureConcat slightly dominates on ModelNet40. Hierarchical SAGPool, although very lightweight, exhibits the lowest accuracy, confirming that the additional expressiveness of the fusion approaches is important for capturing the fine-grained geometric structure of point cloud data.

B. Summary

This research explored point cloud classification by incorporating graph-based representations and node centralities. Two baseline approaches, PointNet and SAGPool, were employed as the basis for investigation. We developed and evaluated two fusion architectures, FeatureConcat and PointNetBasedGraphPooling, which combine features from PointNet and SAGPool.

Feature engineering improved the representation of point cloud data. By building graphs and calculating node centralities—such as betweenness, closeness, Katz, PageRank, eigenvector, harmonic, and load centralities—the structural relationships of each point were explicitly represented. The addition of these connectivity features improved classification accuracy over using raw coordinates alone.

The proposed fusion approaches, FeatureConcat and PointNetBasedGraphPooling, showed significant improvements in classification performance. By

integrating point-wise features extracted by PointNet with graph-level features from SAGPool, the models achieved higher accuracy. As reported in our results, FeatureConcat achieved a test accuracy of 94.34% on ModelNet10, while PointNetBasedGraphPooling achieved the highest accuracy of 95.80%.

In conclusion, combining graph-based pooling and centrality metrics with PointNet’s feature extraction capabilities yields improved representations for point cloud classification. The proposed fusion approaches demonstrate clear performance gains over the individual baselines, providing an effective method for analyzing 3D point cloud data.

C. Future works

- **Diverse Dataset Exploration:** To further validate the robustness and generalization of the proposed fusion approaches, it is essential to explore and evaluate their performance on different datasets with varying complexities and sizes. Investigating datasets that encompass a wider range of 3D objects and scenes will provide deeper insights into the approaches’ adaptability and effectiveness in real-world scenarios.
- **Extended Training Analysis:** In future research, the effect of extended training periods should be explored to understand how longer training durations impact the convergence and overall performance of the fusion approaches. Additionally, investigating the influence of different batch sizes and learning rates on training dynamics and model performance will help in optimizing the training process and achieving higher accuracy.
- **Fine-tuning of Fusion Approach Components:** The proposed fusion approaches, FeatureConcat and PointNetBasedGraphPooling, utilize MLP and 1D convolution layers, dropout, and batch normalization. Fine-tuning these components and hyperparameters can further enhance the network’s performance and efficiency. Optimizing these details will lead to improved feature extraction and pooling, contributing to better classification results. The source code for the proposed fusion approaches, along with the experimental setup and dataset preparation, is available at the this link. Researchers are encouraged to contribute to the development and completion of this work. Collaborating on refining the fusion approaches, exploring additional datasets, and conducting in-depth experiments will enhance the credibility and significance of the findings.

References

- [1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 652–660.
- [2] J. Lee, I. Lee, and J. Kang, “Self-Attention Graph Pooling,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 3734–3743.
- [3] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017, pp. 5099–5108.
- [4] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph CNN for learning on point clouds,” *ACM Trans. Graph.*, vol. 38, no. 5, art. no. 146, pp. 1–12, Oct. 2019.
- [5] M. Defferrard, X. Bresson, and P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, Barcelona, Spain, 2016, pp. 3844–3852.
- [6] D. K. Hammond, P. Vandergheynst, and R. Gribonval, “Wavelets on graphs via spectral graph theory,” *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, Mar. 2011.
- [7] U. von Luxburg, “A tutorial on spectral clustering,” *Stat. Comput.*, vol. 17, no. 4, pp. 395–416, Dec. 2007.
- [8] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, “The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains,” *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, May 2013.
- [9] J. Lee, I. Lee, and J. Kang, “Self-attention graph pooling,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 3734–3743.
- [10] H. Gao and S. Ji, “Graph U-Nets,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 2083–2092.
- [11] Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen, “PointCNN: Convolution on X-transformed points,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Montréal, QC, Canada, 2018, pp. 828–838.
- [12] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, “KPConv: Flexible and deformable convolution for point clouds,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Seoul, South Korea, 2019, pp. 6411–6420.
- [13] L. C. Freeman, “Centrality in social networks: Conceptual clarification,” *Social Networks*, vol. 1, no. 3, pp. 215–239, 1978.
- [14] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford Univ. Press, 2010.